Code analysis: Before optimization. The isPrime function is taking forever.

```
● → jjohnson7 git:(develop) ✗ python JohnsonJeffPA7.py
[1049, 3299, 2521, 3739, 1759, 569, 4657, 4241, 4969, 2657]
         31535 function calls in 89.398 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000   89.398   89.398 <string>:1(<module>)
        1    0.001    0.001   89.398   89.398 JohnsonJeffPA7.py:13(findPrimes)
     3668    0.001    0.000    0.004    0.000 JohnsonJeffPA7.py:3(guess)
     3668   89.392    0.024   89.392    0.024 JohnsonJeffPA7.py:6(isPrime)
     3668    0.001    0.000    0.002    0.000 random.py:245(_randbelow_with_getrandbits)
     3668    0.001    0.000    0.004    0.000 random.py:335(randint)
     7336    0.000    0.000    0.000    0.000 {built-in method _operator.index}
        1    0.000    0.000   89.398   89.398 {built-in method builtins.exec}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.print}
     3668    0.000    0.000    0.000    0.000 {method 'bit_length' of 'int' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
     5854    0.001    0.000    0.001    0.000 {method 'getrandbits' of '_random.Random' objects}
```

```
● → jjohnson7 git:(develop) ✗ python JohnsonJeffPA7.py
[4007, 1811, 4639, 4421, 569, 4441, 389, 1283, 4549, 2029]
         31848 function calls in 0.006 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.006    0.006 <string>:1(<module>)
        1    0.001    0.001    0.006    0.006 JohnsonJeffPA7.py:14(findPrimes)
     3693    0.000    0.000    0.004    0.000 JohnsonJeffPA7.py:3(guess)
     3693    0.002    0.000    0.002    0.000 JohnsonJeffPA7.py:6(isPrime)
     3693    0.001    0.000    0.002    0.000 random.py:245(_randbelow_with_getrandbits)
     3693    0.001    0.000    0.003    0.000 random.py:335(randint)
     7386    0.000    0.000    0.000    0.000 {built-in method _operator.index}
        1    0.000    0.000    0.006    0.006 {built-in method builtins.exec}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.print}
     3693    0.000    0.000    0.000    0.000 {method 'bit_length' of 'int' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
     5992    0.000    0.000    0.000    0.000 {method 'getrandbits' of '_random.Random' objects}
```

Old code:

```
def isPrime(x):
    for i in range(x):
        for j in range(x):
            if i * j == x:
                return False
    return True
```

Code optimization:

```
def isPrime(x):
    if x < 2:
        return False
    for i in range(2, int(x ** 0.5) + 1):
        if x % i == 0:
            return False
    return True
```

Analysis and optimization report:
The original isPrime function was slow because it checked every possible pair of numbers to see if its product was equal to x. That means it had to do O(xˆ2) checks. To make it faster, instead of checking every pair, we just check if x can be divided evenly by any number from 2 up to its square root. This works because if a number has a factor larger than its square root, it must also have a smaller one that we already checked. That cuts the work down to about O(sqrt(x)) checks, making it much faster.